Rebuild Guide — MYRRHA QPLANT Cryogenic Leak Rate Dashboard v4.0.0

> Step-by-step instructions for idempotent rebuild from source.
> Running the same steps twice produces identical outputs (SHA256-verified).

Prerequisites

Requirement	Version	Notes
Python	3.11+	Tested with 3.11.6
pip	22+	For dependency installation
bash	4+	For shell scripts
git	2.30+	For version control
OS	Linux (Ubuntu 22.04+)	Also works on macOS with bash

Python Packages (auto-installed by `setup.sh`)

Package	Version	Purpose
numpy	1.24.3+	Numerical computing
pandas	2.0.2+	Data manipulation
plotly	5.14.1+	Interactive visualizations
scipy	1.10.1+	Scientific computing
pytest	7.3.1+	Test framework
pytest-cov	4.1.0+	Coverage reporting
PyYAML	6.0.2+	YAML parsing (SSoT)

Step 1: Setup (`./setup.sh`)

```
./setup.sh
```

What it does:
1. Creates Python virtual environment (`venv/`)
2. Installs all dependencies from `requirements.txt`
3. Runs initial build if source files are present
4. Generates `docs/manifest.json` with build metadata
5. Updates `docs/index.html` landing page

Expected output:

```
✓ Virtual environment created
✓ Dependencies installed
✓ Initial build complete
✓ Manifest updated
```

Duration: ~30 seconds (first run), ~5 seconds (subsequent)

Idempotency: Safe to run multiple times. Skips venv creation if already exists. Only reinstalls packages if `requirements.txt` hash has changed.

Step 2: Build (`./build.sh`)

```
./build.sh
```

What it does:
1. Activates virtual environment
2. Runs `src/build_all.py` :
- Executes `src/generate_dashboard.py` (core calculations, plots, HTML pages)
- Executes `src/build_handover.py` (formal handover documents)
- Builds `OUTPUT_MANIFEST.json` (SHA256 hashes of all outputs)
3. Runs `src/build_dashboard.py` (updates `docs/index.html`)
4. Runs `src/generate_visuals_v3.py` (22 Plotly charts)
5. Runs `src/generate_standards_stats.py` (standards & statistics)
6. Updates `docs/manifest.json`

Expected outputs:

```
docs/
├── index.html              (updated)
├── index_v4_0.html         (updated)
├── dashboard.html          (updated)
├── calculations.html       (updated)
├── executive_summary.html  (updated)
├── rtm_traceability.html   (updated)
├── handover.html + .pdf    (updated)
├── plots/*.html            (5 charts)
├── visualizations_v3/*.html (22 charts)
├── manifest.json           (updated)
outputs/
├── tables/*.csv + *.json
├── plots/*.html
OUTPUT_MANIFEST.json        (updated)
```

Duration: ~60–90 seconds

Idempotency: All file writes use `write_text_if_changed()` — files are only written when content differs from existing file (SHA256-checked). Running twice produces zero file changes on the second run.

Step 3: Validate (`./validate.sh`)

```
./validate.sh
```

What it does:
1. Activates virtual environment
2. Runs `pytest tests/ -v` (22 tests)
3. Generates coverage report (`htmlcov/`)
4. Generates test report (`dist/test-report.html`)
5. Generates machine-readable results (`dist/test-results.json` , `dist/junit.xml`)
6. Updates `docs/manifest.json` with test results
7. Refreshes `docs/index.html` with latest test status

Expected output:

```
tests/test_engineering.py::test_dimensional_chain PASSED
tests/test_engineering.py::test_temperature_pressure_scaling PASSED
tests/test_engineering.py::test_reference_rtm048_nm3_day_to_kg_year PASSED
tests/test_engineering.py::test_baseline_scenario_total_range PASSED
tests/test_engineering.py::test_unit_conversion_constant PASSED
tests/test_calculations.py::... PASSED
tests/test_config_loader.py::... PASSED
tests/test_data_integrity.py::... PASSED
tests/test_build_outputs.py::... PASSED
tests/test_outputs.py::... PASSED

========================= 22 passed in ~1.5s =========================
```

Duration: ~5 seconds

Step 4: Package (`./package.sh`)

```
./package.sh
```

What it does:
1. Creates `dist/` directory
2. Bundles project files into `dist/handover.zip`
3. Generates `dist/handover.zip.sha256` for integrity verification
4. Copies test reports to `dist/`

Expected outputs:

```
dist/
├── handover.zip          (~2.7 MB)
├── handover.zip.sha256   (checksum)
├── test-report.html
├── test-results.json
├── junit.xml
└── build.log
```

Duration: ~10 seconds

Complete Rebuild (Single Command)

```
./setup.sh && ./build.sh && ./validate.sh && ./package.sh
```

Or using Make:

```
make all
```

Total duration: ~2–3 minutes

Verification Steps

After rebuild, verify the following:

A. Tests pass

```
source venv/bin/activate
python -m pytest tests/ -v
# Expected: 22 passed
```

B. Build manifest is current

```
python3 -c "
import json
m = json.load(open('docs/manifest.json'))
print(f'Status: {m[\"build\"][\"status\"]}')
print(f'Tests:  {m[\"tests\"][\"summary\"]}')
print(f'Commit: {m[\"build\"][\"git_commit\"]}')
"
# Expected: Status: verified, Tests: 22/22 passed
```

C. Key files exist

```
for f in docs/index.html docs/index_v4_0.html docs/manifest.json \
         docs/STAKEHOLDER_PRESENTATION.html docs/dashboard.html \
         docs/plots/plot1_leak_vs_loss.html OUTPUT_MANIFEST.json; do
  [ -f "$f" ] && echo "✓ $f" || echo "✗ MISSING: $f"
done
```

D. SSoT config loads correctly

```
source venv/bin/activate
python3 -c "
from src.config_loader import load_config
cfg = load_config()
print(f'HP compressors: {cfg.compressors.hp.count}')
print(f'HP outlet:      {cfg.compressors.hp.outlet_pressure_barg} barg')
print(f'Motor power:    {cfg.compressors.hp.motor_power_kw} kW')
"
# Expected: HP compressors: 3, HP outlet: 14 barg, Motor power: 315 kW
```

E. Zip integrity

```
cd dist && sha256sum -c handover.zip.sha256
# Expected: handover.zip: OK
```

Troubleshooting

Problem: `ModuleNotFoundError: No module named 'yaml'`

Fix: Run `./setup.sh` to install dependencies, or:

```
source venv/bin/activate
pip install PyYAML==6.0.2
```

Problem: `venv/bin/activate: No such file or directory`

Fix: Run `./setup.sh` first to create the virtual environment.

Problem: Tests fail with import errors

Fix: Ensure you're running from the project root:

```
cd /path/to/cryo_leak_rate_dashboard
source venv/bin/activate
python -m pytest tests/ -v
```

Problem: `weasyprint` errors during handover PDF generation

Fix: WeasyPrint requires system-level libraries:

```
sudo apt-get install -y libpango-1.0-0 libpangocairo-1.0-0 libgdk-pixbuf2.0-0
pip install weasyprint
```

Note: PDF generation is optional — HTML handover is always generated.

Problem: Plotly charts show blank in browser

Fix: Charts are standalone HTML files with embedded JavaScript. Open with a modern browser (Chrome, Firefox, Edge). If using `file://` protocol, some browsers block local JS — use a local server:

```
python -m http.server 8000
# Then open http://localhost:8000/docs/index.html
```

Problem: `permission denied` on shell scripts

Fix:

```
chmod +x setup.sh build.sh validate.sh package.sh
```

Problem: Build produces different SHA256 hashes

Cause: Timestamps in generated HTML files change on each build. The idempotent write system (`src/manifest.py`) handles this for content — but manifest timestamps will differ.
Verification: Compare content hashes, not file timestamps.

Problem: Git shows unexpected changes

Fix: Check `.gitignore` is correctly configured:

```
git status --porcelain
# Generated files in docs/, outputs/, dist/ should be tracked
# venv/, __pycache__/, .coverage should be ignored
```

Architecture Notes for Developers

Data Flow

```
data/config.yaml (SSoT)
    ↓
src/config_loader.py (loads + validates)
    ↓
src/calc_leak_rate.py (physics calculations)
    ↓
src/generate_dashboard.py (plots + HTML + tables)
    ↓
docs/ (HTML pages, Plotly charts, CSV/JSON exports)
```

Adding a New Parameter

1. Add to `data/config.yaml`
2. Update `src/config_loader.py` if schema changes

3. Reference via config loader in calculation modules
4. Add test in `tests/test_config_loader.py`
5. Run `./build.sh && ./validate.sh`

Adding a New Visualization

1. Add plot function in `src/generate_dashboard.py` or `src/generate_visuals_v3.py`
2. Save to `docs/plots/` or `docs/visualizations_v3/`
3. Update `docs/index_v4_0.html` if adding to navigator
4. Run `./build.sh && ./validate.sh`

Adding a New Test

1. Create test function in appropriate `tests/test_*.py` file
2. Run `./validate.sh` to confirm
3. Expected test count increases accordingly
